

Avery Harding

Section 02

Question 1:

Steps:

None

```
h(state, goal):
    if state == goal:
        return 0
    h = 0
    for i in range(len(state)):
        if state[i] != goal[i]:
            h += 1
    return h
```

This heuristic lower-bounds the lowest number of character replacements needed to match the current state with the goal state. It is admissible because, since each step has a cost of 1, the total cost to replace each character in the current state cannot be less than the number of operations.

Scrabble:

None

```
h(state, goal):
    if state == goal:
        return 0
    h = 0
    for i in range(len(state)):
        if state[i] != goal[i]:
            h = h + scrabble_costs[goal[i]]
    return h
```

This heuristic lower-bounds the required Scrabble cost of replacing each goal character. Because each transition step incurs a Scrabble cost greater than or equal to 1, the summation of the minimum costs to replace each letter cannot exceed the true optimal path cost.

Frequency:

None

```
h(state, goal):
    if state == goal:
        return 0

    h = 0
    for i in range(len(state)):
        if state[i] != goal[i]:
            h += 1

    return h + rarity(goal)
```

This heuristic lower-bounds the minimum number of character replacements needed to reach the goal state. Adding 1 + rarity value to the path cost only incurs a non-negative additional cost. Thus, the minimum number of character replacements will not exceed the true optimal path cost.

Question 2:

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY ->ASK	Steps	5	773	5	16	97.9301423
WHY ->ASK	Scrabble	9	765	9	13	98.30065359
WHY ->ASK	Frequency	38.85540652	808	38.85540652	634	21.53465347
BEAR -> DUCK	Steps	4	691	4	6	99.1316932
BEAR -> DUCK	Scrabble	11	2981	11	6	99.79872526
BEAR -> DUCK	Frequency	38.2402284	1084	38.2402284	302	72.1402214

1. UCS and A* always returned the same optimal cost.
2. A* produced the largest reduction under the Scrabble cost measure for both word pairs.
3. The frequency cost heuristic is the least informative.
4. The frequency cost heuristic only accounts for the minimum step cost of each transition. The Scrabble cost heuristic depends on a specific cost value of each replaced letter.

Question 3:

Steps:

1. Yes
2. For each word, each action has the same cost. W_1 -> W_2 has the same number of actions as W_2 -> W_1

Scrabble:

1. No
2. Each action's cost is determined by the specific value of the new letter. When words are reversed a different letter, which may have a different Scrabble value, takes the original letter's position.
3. dog -> cog: cost = 3 | cog -> dog: cost = 2

Frequency:

1. No
2. When words are reversed, two words with a different rarity value cause the action cost to be asymmetric
3. dog -> cog: cost = 1 + rarity(cog) | cog -> dog: cost = 1 + rarity(dog)

Question 4:

1. Model: Gemini 3.6 Thinking. Proposed rarity-aware Hamming distance heuristic.
2. $h(n) = 0$ if $n = g$, $h(n) = \{d_H(n, g) + \text{rarity}(g) + (d_H(n, g) - 1) \cdot r_{\min}\}$ if $n \neq g$
where $d_H(n, g)$ denotes Hamming distance between state n and goal state g and $r_{\min}$ denotes the global minimum rarity across all words in the dictionary.
3. By relaxing the problem, it can be determined if this $h(n)$ is admissible.

Relaxed constraints:

1. Instead of all transition words being required to be valid dictionary words, any single-letter character replacement is valid.
2. Minimum cost for each transition state: $1 + r_{\min}$

By adding the minimum transition cost for each step to the cost of the final step, $1 + \text{rarity}(g)$,

$$h_{\text{relaxed}}(n) = (d_H(n, g) - 1)(1 + r_{\min}) + (1 + \text{rarity}(g))$$

$$h_{\text{relaxed}}(n) = (d_H(n, g) + \text{rarity}(g) + (d_H(n, g) - 1)r_{\min}) = h(n)$$

Word pair	A* cost $h_0(n)$	A* nodes expanded $h_0(n)$	A* cost $h(n)$	A* nodes expanded $h(n)$
WHY -> ASK	38.85540652	755	38.85540652	634
BEAR -> DUCK	38.2402284	720	38.2402284	301

Compared to the baseline heuristic, the heuristic given by the LLM results in the same total cost, but fewer nodes expanded, especially with 4 letter words. Thus, the LLM's heuristic is more efficient.